NYCU-EE IC LAB – Fall 2026

Lab03 Exercise

Design: Tree Harvester

Data Preparation

1. Extract files from TA's directory:

   % tar xvf ~iclabTA01/Lab03_2026spring.tar

2. The extracted LAB directory contains:
   a. **00_TESTBED**
   b. **01_RTL**
   c. **02_SYN**
   d. **03_GATE**

Introduction

Welcome to the Data Forest!

In the world of Big Data, information is often stored in complex, hierarchical structures (Trees) scattered across vast memory spaces. To make use of this data, we need a specialized hardware accelerator are required to perform two critical tasks: Extraction and Organization.

In this lab, you will build the **Tree Harvester**, an intelligent memory controller designed to operate on a **DRAM** forest. Your design will perform the following core operations:

1. **The Harvest (CALC Mode):** The DRAM is populated with **Prefix Expression Trees**. These trees represent raw arithmetic operations. Your Harvester must traverse these pointer-based structures, compute their values using a 64-bit ALU, and extract the final result.

2. **The Sorting (SORT Mode):** The harvested fruits (data) are scattered haphazardly across different banks. You must gather these fruits together and "sorting" them according to their quality (numerical value) before they can be shipped out.

While implementing the correct CALC and SORT algorithms is essential, the primary bottleneck of this system is memory access latency. A **naive interface** implementation that accesses **DRAM** sequentially (one bank at a time) will suffer from significant idle cycles due to DRAM physical constraints (Row activation, Precharge, etc.). To pass the strict latency requirements of this Lab and maximize system throughput, your design may utilize **Bank Interleaving**. By overlapping the

operation phases of different banks (e.g., activating Bank 1 while Bank 0 is waiting for data), you can maximize the bus utilization and achieve high throughput. You will act as an **AXI4-Lite Master** to issue requests, and implement a **DRAM Controller (AXI Slave)** to translate these requests into physical DRAM commands, handling all timing constraints (t_RCD, t_CL, t_RP).

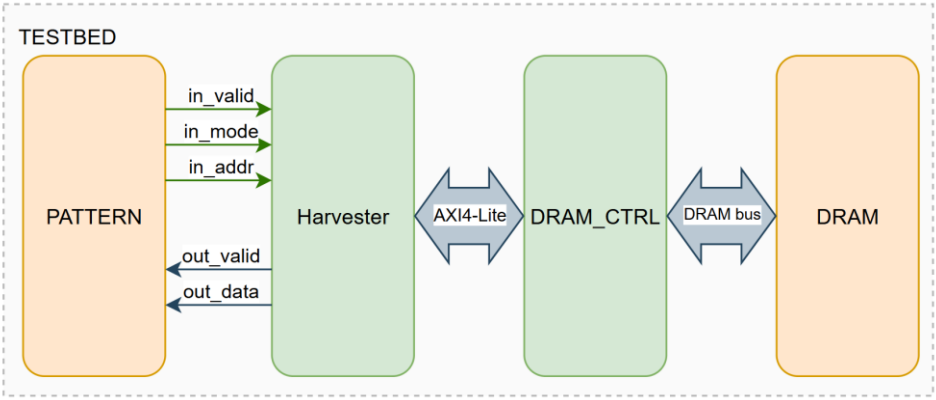


Fig.1 System diagram

The system consists of four main modules:

- **PATTERN :** Generates commands (READ, WRITE, CALC, SORT) and verifies results.
- **HARVESTER(Master):** Your RTL design. It acts as an AXI4-Lite Master to access DRAM and contains the arithmetic and sorting logic.
- **DRAM_CTRL(Slave):** Your RTL design. An interface module that translates AXI4-Lite requests into physical DRAM commands (RAS/CAS/WE) while adhering to timing constraints.
- **pseudo DRAM:** A simulation model storing the Prefix Expression Trees and Data Arrays.

In this lab, your task is to finalize the **PATTERN**. We will supply both encrypted versions of correct and incorrect designs to guide and assist you in this process. Additionally, we offer a pseudo DRAM that not only simulates the behavior of actual DRAM but also verifies its specifications. Once the PATTERN and pseudo DRAM are complete, your next step is to design DRAM_CTRL connecting the DRAM and the Harvester.

Problem Description(HARVESTER.v)

DRAM Memory Map

The DRAM address space (65,536 words, 64-bit each word) with 4 Banks is partitioned as following field:

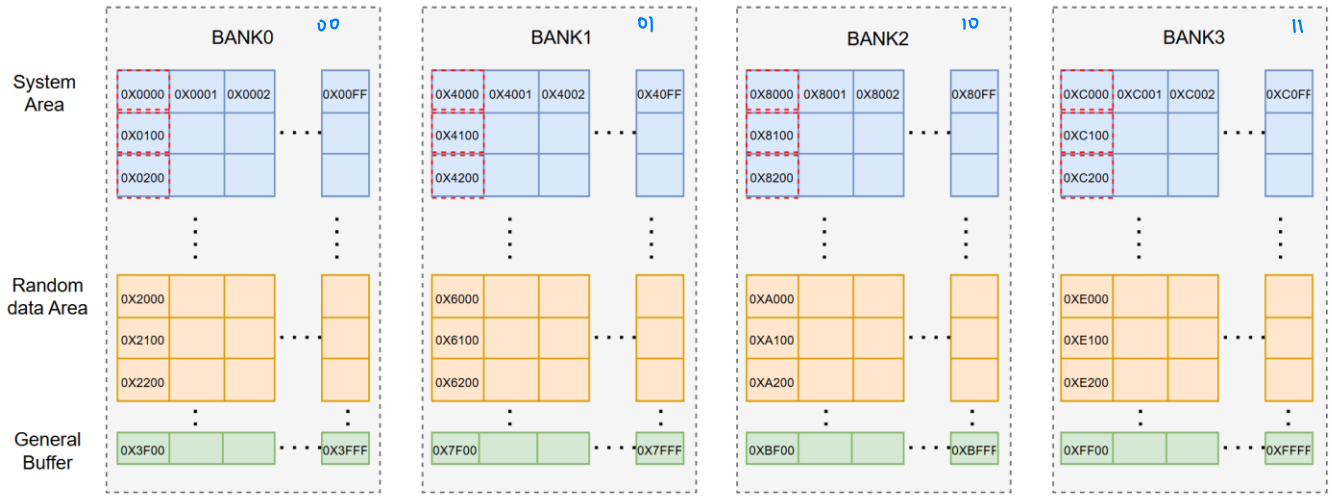


Fig .2 DRAM address map

Row Range	Address Range(Hex)	Description
0~31	B0:0x0000 ~ 0x1FFF B1:0x4000 ~ 0x5FFF B2:0x8000 ~ 0x9FFF B3:0xC000 ~ 0xDFFF	System area for **Prefix Expression Trees**, which is read only.
32 ~ 62	B0:0x2000 ~ 0x3EFF B1:0x6000 ~ 0x7EFF B2:0xA000 ~ 0xBEFF B3:0xE000 ~ 0xFEFF	Read/Write Random Data Area.
63	B0:0x3F00 ~ 0x3FFF B1:0x7F00 ~ 0x7FFF B2:0xBF00 ~ 0xBFFF B3:0xFF00 ~ 0xFFFF	General purpose buffer without check with golden by pattern

DRAM could be access with 4 Banks (64 rows each bank,256 columns each row).

Example: 0x410E is B1_R1_C14.

Operation Modes

This Lab contain two **basic operation** and two **cross bank operation**.The operation is determined by the in_mode signal:

- **READ (2'd0) :** Read data from DRAM in_src_row and output to out_data.
- **WRITE (2'd1) :** Write in_data to DRAM in_dst_row.
- **CALC (2'd2) :** Solve the Prefix Expression Trees stored in DRAM.
- **SORT (2'd3) :** Sort in_src_row for four banks (1024 items) and Write to DRAM in_dst_row.

Please note that you should calculate the golden answer in the PATTERN. The golden answer from external files is NOT allowed.

Detailed Specification:

- **READ Operation :**
  1. **Task:** You must read out the whole row data at the address specify by in_bank 、in_src_row(range from 0~62).
  2. **Output:** Output the result of each column data via out_data with out_valid **256 cycles** sequentially.
- **WRITE Operation :**
  1. **Task:** You must write the data input (64 bits in_data) by in_data at the address specify by in_bank 、in_dst_row(range from 32~62).
  2. **Output:** Pull high out_valid **1 cycle** and PATTERN would check DRAM memory with golden immediately.

- **CALC Operation :**

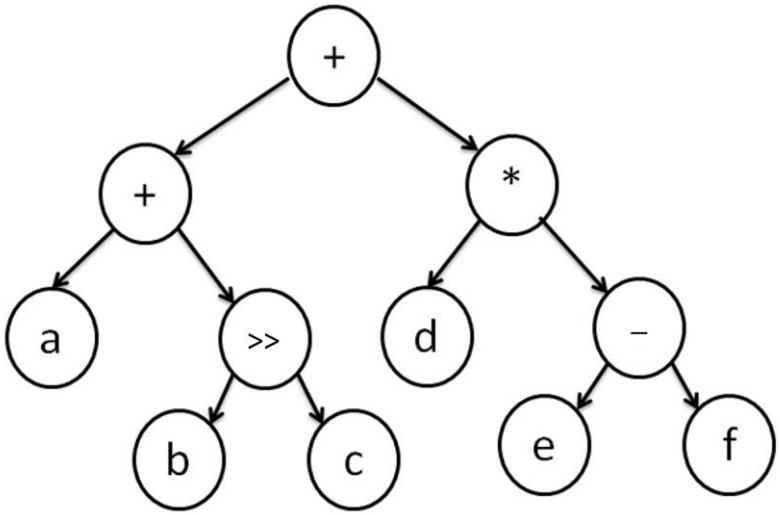


Fig.3 Prefix Expression Tree . The equation is {a+(b>>>c[5:0])+d*(e-f)}.

1. **Task:** You must process 4 trees sequentially or in parallel (one from each Bank: 0, 1, 2, 3).
2. **Root Location:** The root node of the tree is always located at **Column 0** of the pattern provides in_src_row(range from 0~31) for 4 banks. The prefix expression tree supports a **maximum depth of 8 levels** and a **maximum capacity of 255 nodes**. The root location can be referenced from the red box in fig 1.
3. **Arithmetic Rules:**
   - All calculations are **64-bit Signed**.
   - Pattern would exclude invalid arithmetic result like:
     - **Overflow:** Result of ADD/SUB/MULT exceeds 64-bit signed range.

4. **Format:** Each 64-bit word in the **System Area** (ROW 0~31)represents a tree node. The data type depends on the **Mode Bit [63]**.
   - **Address pointer:** 16-bit Absolute Address {Bank[1:0], Row[5:0], Col[7:0]}.
   - **Word format:**

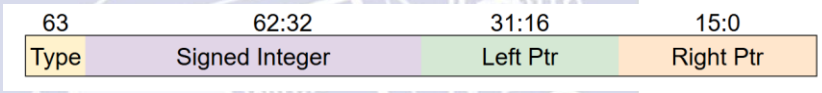


Fig.2 Number format(**Type 0**)

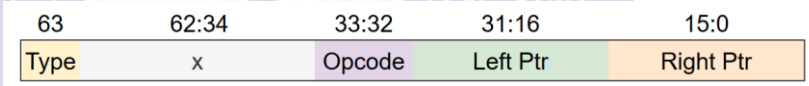


Fig.3 Operator format(**Type 1**)

Note: For Number nodes (Type 0), the Left Ptr and Right Ptr fields are invalid and should be ignored (or contain 0xFFFF).

   - **Opcode:**
     a. 2'b00: **ADD (+)**
     b. 2'b01: **SUB (-)**
     c. 2'b10: **MULT (*)**
     d. 2'b11: **Aritjmetic Shift Right(>>>) :** Left child [63:32] >>> Right child [37:32])

Signed bit 要 填 回 去

5. **Output:** Output the result of each tree via out_data with out_valid **4 cycles** sequentially.The outputs must strictly follow the order: Result of Bank 0 → Bank 1 → Bank 2 → Bank 3.

◆ **SORT Operation:**

1. **Source:** in_src_row(range from 0~62) contains data distributed across **4 Banks**.
   - Each Bank holds 256 words (Column 0 to 255).
   - Total: **1024 words**.
2. **Task** :Compare **the data field [62:32] (31 bits)**of 1024 words and sort them in Ascending Order. The sorted data (1024 words) must be written back across all 4 banks at the specified in_dst_row (i.e., B0_C0~255 → B1_C0~255 → B2_C0~255 → B3_C0~255). If two data entries have the same value, the one with the **smaller address is sorted first**.
3. **Output** :Pull high out_valid **1 cycle**, and pattern would check DRAM memory if match with golden.

AXI4-Lite Protocol (Master: HARVESTER .v ⇔ Slave: DRAM_CTRL.v)

For simplicity, we use AXI4-Lite protocol to access DRAM. You may utilize outstanding algorithm for higher throughput.

Global Signals

Signal	Bit Width	Source	Description
clk	1	Clock source	Global clock signal. All signals are sampled on the **Positive** edge of the global clock.
rst_n	1	Reset source	Global reset signal. This signal is active LOW.

Write Address Channel

Signal	Bit Width	Source	Description
aw_addr	32	Master	Write address.
aw_valid	1	Master	Write address valid. This signal indicates that valid write address is available: 1 = address available 0 = address not available. The address remain stable until the address acknowledge signal, **AW_READY**, goes HIGH.
aw_ready	1	Slave	Write address ready. This signal indicates that the slave is ready to accept an address signals: 1 = slave ready 0 = slave not ready.

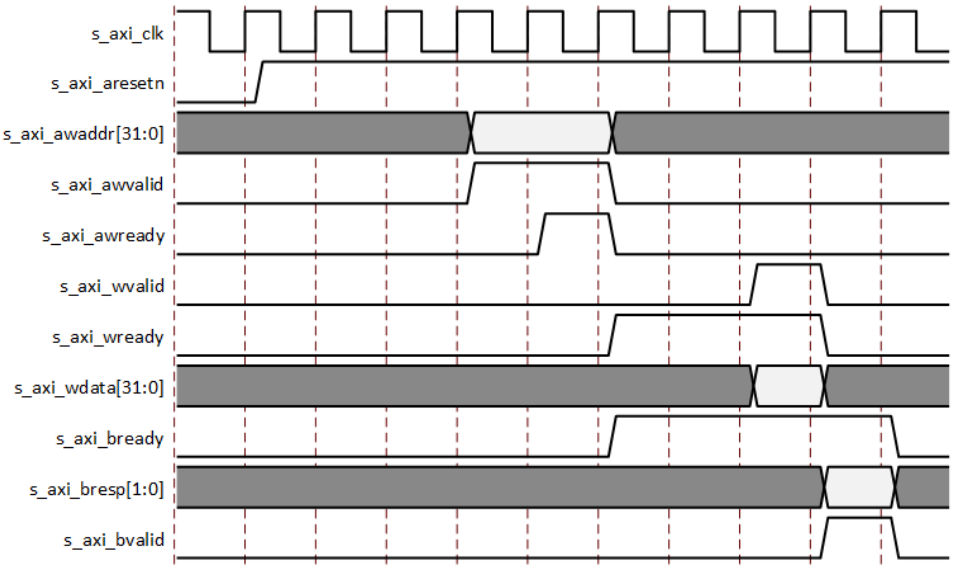
Write Data Channel

Signal	Bit Width	Source	Description
w_data	64	Master	Write data bus.
w_valid	1	Master	Write valid. This signal indicates that valid write data is available: 1 = write data available 0 = write data not available.
w_ready	1	Slave	Write ready. This signal indicates that the slave can accept the write data: 1 = slave ready 0 = slave not ready.

Write Response Channel

Signal	Bit Width	Source	Description
b_resp	2	Slave	Write response. This signal indicates the status of the write transaction. The allowable responses are OKAY, EXOKAY, SLVERR, and DECERR. (In this lab we only issue OKAY(2'b00))
b_valid	1	Slave	Write response valid. This signal indicates that a valid write response is available: 1 = write response available. 0 = write response not available.
b_ready	1	Master	Response ready. This signal indicates that the master can accept the response information. 1 = master ready. 0 = master not ready.

Waveform of Write Transaction



(AMD Adaptive Computing Documentation Portal)

Spec about Write Operation

- Outstanding Capability:

The Master is allowed to send multiple AW and W messages consecutively before receiving B_VALID.

- AW & W Dependency:
  - For this lab's simplification, the Master must ensure that AW_VALID goes high earlier than or equal to W_VALID.
  - The Master cannot send W_data without a corresponding AW. (Dependency Check)
  - The Slave will only go high W_READY after receiving AW (or simultaneously).
- B Channel (Response):
  - Because Burst is not supported, the Slave must send back B_VALID after each one-word write.
  - The Master must be ready to receive B_VALID at any time (it is recommended to set B_READY directly to 1).

1. After **AW_VALID** is high, **AW_READY** should be pulled high in 0~50 cycles.(Timeout Check1)
2. **AW_VALID** and **AW_ADDR** should remain stable until **AW_READY** goes high. (Stability Check)
3. **AW_ADDR** should be within the legal range (0~65535).
4. **AW_ADDR** should be valid when **AW_VALID** is high.
5. **AW_ADDR** should be reset when **AW_VALID** is low.
6. After write address channel communicate, **W_VALID** and **W_READY** should be pulled high in 0~150 cycles. (Timeout Check2)
7. After **W_VALID** is high, **W_READY** should be pulled high in 0~100 cycles.(Timeout Check1)
8. **W_DATA** should be valid when **W_VALID** is high.
9. **W_DATA** should be reset when **W_VALID** is low.
10. **W_VALID** and **W_DATA** should be remained stable until **W_READY** goes high. (Stability Check)
11. After write data channel communicate, **B_VALID** should be pulled high in 0~150 cycles. (Timeout Check2)
12. After **B_VALID** is high, **B_READY** should be pulled high in 0~100 cycles.(Timeout Check1)
13. **B_RESP** should be valid when **B_VALID** is high.
14. **B_RESP** should be reset when **B_VALID** is low.
15. **B_VALID** and **B_RESP** should be remained stable until **B_READY** goes high. (Stability Check)

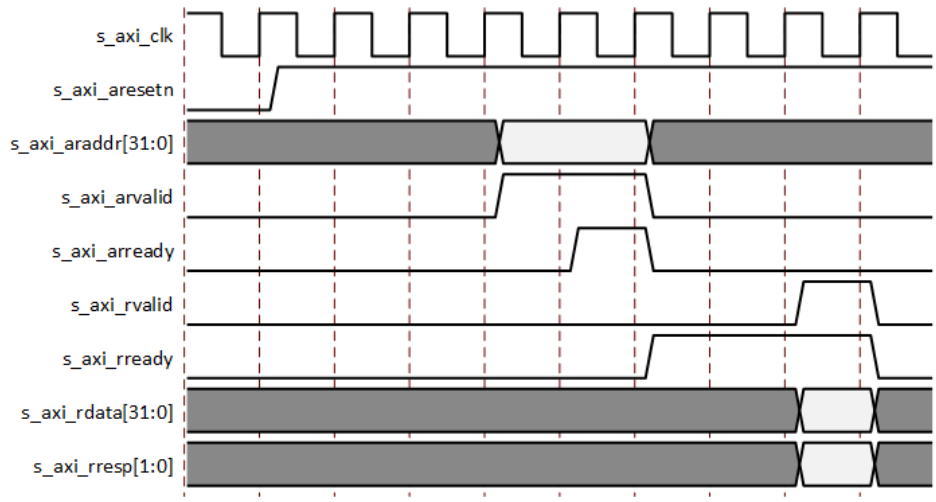
Read Address Channel

Signal	Bit Width	Source	Description
ar_addr	32	Master	Read address.
ar_valid	1	Master	Read address valid. This signal indicates, when HIGH, that the read address is valid and will remain stable until the address acknowledge signal, **AR_READY**, is high. 1 = address and control information valid 0 = address and control information not valid.
ar_ready	1	Slave	Read address ready. This signal indicates that the slave is ready to accept an address signal: 1 = slave ready 0 = slave not ready.

Read Data Channel

Signal	Bit Width	Source	Description
r_data	64	Slave	Read data.
r_resp	2	Slave	Read response. This signal indicates the status of the read transfer. The allowable responses are OKAY, EXOKAY, SLVERR, and DECERR. (In this project we only issue OKAY(2'b00))
r_valid	1	Slave	Read valid. This signal indicates that the required read data is available, and the read transfer can complete: 1 = read data available 0 = read data not available.
r_ready	1	Master	Read ready. This signal indicates that the master can accept the read data and response information: 1= master ready 0 = master not ready.

Waveform of Read Transaction



(AMD Adaptive Computing Documentation Portal)

Spec about Read Operation

- Outstanding Capability:
  - The Master allows multiple AR requests to be sent consecutively without waiting for the previous R_DATA to return.
- In-Order Response:
  - The response order must match the request order.
  - If the Master sends AR(Addr_A) -> AR(Addr_B) in that order,
  - the Slave must respond with R(Data_A) -> R(Data_B) in that order.

1. After **AR_VALID** is high, **AR_READY** should be pulled high in 0~50 cycles. (Timeout Check1)
2. **AR_VALID** and **AR_ADDR** should remain stable until **AR_READY** goes high. (Stability Check)
3. **AR_ADDR** should be within the legal range (0~65535).
4. **AR_ADDR** should be valid when **AR_VALID** is high.
5. **AR_ADDR** should be reset, when **AR_VALID** is low.
6. After read address channel communicate, **R_VALID** and **R_READY** should be pulled high in 0~150 cycles. (Timeout Check2)
7. After **R_VALID** is high, **R_READY** should be pulled high in 0~100 cycles. (Timeout Check1)
8. **R_DATA** should be valid when **R_VALID** is high.
9. **R_DATA** should be reset when **R_VALID** is low.
10. **R_VALID**、**R_DATA** and **R_RESP** should be remained stable until **R_READY** goes high. (Stability Check)

Physical DRAM bus (DRAM_CTRL.v ⇔ pseudo_DRAM.v)

Physical DRAM Interface

The pseudo_DRAM module models a simplified DRAM chip with 4 Banks. It does **not** speak AXI. Your DRAM_CTRL module must function as a bridge:

- **Front-end(DRAM_CTRL):** AXI4-Lite Slave (Receives AW, W, AR from Bridge/Harvester).
- **Back-end(pseudo_DRAM):** Physical DRAM Interface (Drives RAS, CAS, WE, etc. to pseudo_DRAM).

I/O Signals

■ The following are the definition of I/O signals of **DRAM_CTRL.v**

Signal	Bit Width	Direction	Description
clk	1	Input	Global clock signal. All signals are sampled on the **Positive** edge of the global clock.
rst_n	1	Input	Active-low reset.
dram_cmd	4	Output	Combine with **{dram_cs_n,dram_ras_n,dram_cas_n,dram_we_n}**
dram_ba	2	Output	Bank Address (Selects one of 4 banks).
dram_addr	11	Output	Address Input **(Multiplexed Row/Column Address)**.
dram_wdata	64	Output	Write Data.
dram_rdata	64	Input	Read Data (Valid after t_CL).
dram_valid	1	Intput	Read Data Valid (High when dram_rdata is valid).

Command Truth Table

Command	dram_cs_n	dram_ras_n	dram_cas_n	dram_we_n	Description	dram_addr
No Operation (NOP)	L	H	H	H	Idle state. Keeps previous state active.	x
Activate (ACT)	L	L	H	H	Opens a Row in a specific Bank. Requires dram_ba (Bank) and dram_addr (Row).	Row Address

Read (READ)	L	H	L	H	Initiates a Read burst. Requires dram_ba (Bank) and dram_addr (Col).	Column Address
Write (WRITE)	L	H	L	L	Initiates a Write burst. Requires dram_ba, dram_addr (Col), and dram_wdata.	Column Address
Precharge (PRE)	L	L	H	L	Closes the open Row in a Bank. Requires dram_ba.	x

Operation Sequence

To access a specific memory location defined by {Bank, Row, Column}, the controller must follow a strict state sequence.

◆ **Address Muxing**

The dram_addr bus is shared. Your controller must multiplex the address based on the current command:

- **During ACT:** dram_addr = **Row Address**.
- **During READ/WRITE:** dram_addr = **Column Address**.

◆ **Standard Read Cycle**

To read data from the pseudo_DRAM, your controller must follow this sequence:

1. **Activate:** Send ACT command with dram_ba and Row Address.
2. **Wait t_RCD:** Wait 2 clock cycles for the row to open.
3. **Read:** Send READ command with dram_ba and Column Address.
4. **Wait t_CL:** Wait 3 clock cycles for the data to be fetched from the array.
5. **Data Valid:** The dram_valid signal goes HIGH, and the requested 64-bit data is available on dram_rdata.
6. **Precharge:** If the next access is to a different Row in the same Bank, send PRE command, wait t_RP(5 clock cycles after ACT command), then start over.

◆ **Standard Write Cycle**

To write data to the pseudo_DRAM, your controller must follow this sequence:

1. **Activate:** Send ACT command with dram_ba and Row Address.
2. **Wait t_RCD:** Wait 2 clock cycles for the row to open.
3. **Write:** Send WRITE command with dram_ba and Column Address and **simultaneously** drive the 64-bit data onto dram_wdata.
4. **Precharge:** If the next access is to a different Row in the same Bank, send PRE command. Just like the read cycle, you must ensure at least 5 clock cycles have passed since the initial ACT command before issuing the PRE command.

Timing Constraints & Latency

The pseudo_DRAM simulates physical delays. You must strictly adhere to the following timing parameters to ensure data integrity. Violating these constraints may result in ignored commands or corrupted data.

1. **t_RCD** (Row Address to Column Address Delay):
   - Latency: **2 Cycles**
   - Description: After issuing an **ACT** command, you must wait 2 clock cycles before issuing a **READ** or **WRITE** command to the *same* bank.
2. **t_CL** (CAS Latency):
   - Latency: **3 Cycles**
   - Description: After issuing a **READ** command, the valid data (dram_valid high) will appear after **3** clock cycles.
3. **t_RP** (Row Precharge Time):
   - Latency: **3 Cycles**
   - Description: After issuing a **PRE** command, you must wait 3 clock cycles before issuing a new **ACT** command to the *same* bank.
   - *Note:* You only need to Precharge if you want to switch Rows within the same bank.
4. **t_RAS** (Row Active Time):
   - Latency: **5 Cycles**
   - Description: After issuing an **ACT**, the row must remain open for at least 5 cycles before a **PRE** command can be issued.

Note: The pseudo_DRAM allows operations on different banks to overlap. While Bank 0 is busy preparing data (latency period), you can issue a command to Bank 1.

The specific latency values are defined inside the encrypted pseudo_DRAM.v. Your design (Harvester)must be robust enough to handle varying handshake delays compliant with the AXI standard.

Inputs / Outputs (HARVESTER.v)

■ The following are the definition of input signals

Input Signals	Bit Width	Definition
clk	1	Clock
rst_n	1	Asynchronous active-low reset
in_mode_valid	1	High when in_mode signals are valid.
in_mode	2	◆ 2‘b00:READ ◆ 2‘b01:WRITE ◆ 2‘b02:CALC ◆ 2‘b02:SORT
in_valid	1	High when the specific input signals (address or write data)are valid.
in_bank	2	Target bank index (valid when READ/WRITE).
in_src_row	6	Source row address range from 0~62.(valid when READ CALC SORT mode)
in_dst_row	6	Destination Row address for writing results range from 32~62.(valid when WRITE SORT mode)
in_data	64	Data input for WRITE mode.

■ The following are the definition of output signals

Output Signals	Bit Width	Definition
out_valid	1	High when all read and write operation finish and out_data is valid. DRAM memory would be check at the same time.
out_data	64	Computation result or Read data starting from the MSB. Each pattern should have exactly the cycles according to mode. ◆ READ : 256 cycles ◆ WRITE: 1cycles ◆ CALC : 4 cycles ◆ SORT : 1 cycles

1. All inputs will be changed at the clock negative edge.
2. All input signals are synchronized at the **negative edge** of the clock.
3. For each pattern, **in_mode_valid** should keep high for **1** cycle then **in_valid** should keep high for **1/256/1/1** cycles for mode READ/WRITE /CALC/SORT and **out_valid** should keep high for **256/1/4/1** cycles according to mode READ/WRITE /CALC/SORT.

4. The next input pattern will come in **2~4 negative edge of the clock** after your **out_valid** is pulled down.

Specifications

1. Top module name: HARVESTER 、DRAM_CTRL(design file name: HARVESTER.v、DRAM_CTRL.v)
2. It is asynchronous reset and active-low architecture. If you use synchronous reset (considering reset after clock starting) in your design, you may fail to reset signals.
3. All the design including HARVESTER 、 DRAM_CTRL and DRAM are synchronized at the **posedge edge** of the clock, and **operate at the same clock frequency**.
4. The **reset signal (rst_n)** would be given only once at the beginning of simulation. All output signals should be reset . The pattern will check the output signal 200ns after the reset signal is pulled low.
5. The execution latency of each operation is limited in **10,000** cycles**.** The latency is the time of the clock cycles between the **falling edge of the in_valid and the rising edge of the out_ valid**.
6. The **area** of each module is limited in **7,500,000** $\boldsymbol{um^2}$.
7. The **out_valid** and **out_data** must be asserted in **specified cycles**.
8. The **out_data** should be correct when **out_valid** is high.
9. The data in the DRAM should be correct when **out_valid** is high.
10. The maximum **clock period** is set to **40 ns**.
11. The synthesis **ELAPSED TIME** is limited in **1 HOUR**.
12. The input delay is set to **0.5*(clock period)**.
13. The output delay is set to **0.5*(clock period)**, and the output loading is set to **0.05**.
14. The synthesis result of the data type **cannot** include any **latches**.
15. Gate-level simulation **cannot** include any **timing violations** without the notimingcheck command.
16. After synthesis, The slack at the end of the timing report should be **non-negative (MET)**.
17. Any words with “error”, or “congratulation” can’t be used as variable name.

Grading Policy

1. **Test Bench**: 45% (2nd demo **100% off**)

   Demo: Use your pattern.v, use the HARVESTER_encrypted.v, DRAM_CTRL_encrypted.v, pseudo_DRAM.v and DRAM_init.dat we provided.

   Use your pattern.v to check the fllowing specification :

   - (15%) Pattern correctness. (Complete the simulation when design is correct.)
   - (2%) SPEC MAIN-1: All output signals **including AXI signals** should be reset after the

reset signal is asserted.

- (2%) SPEC MAIN-2: The execution latency is limited in 10000 cycles.
- (2%) SPEC MAIN-3: The **out_valid** and **out_data** must be asserted in relative cycles.
- (2%) SPEC MAIN-4: The **out_data** should be correct when **out_valid** is high.
- (2%) SPEC MAIN-5: The data in the DRAM should be correct when **out_valid** is high.

Use your pattern.v to check the fllowing AXI4-Lite specification :

- (3%) SPEC AXI-1: Data、address and response should be reset when valid is low.
- (5%) SPEC AXI-2: The signal of AW W B AR R channel should be stable before receive response.(Stability Check)
- (2%) SPEC AXI-3: The w_valid signal asserted without corresponding aw_vlaid is not allowed.(Dependency Check)
- (2%) SPEC AXI-4: The address should be within the legal range (0~65535).
- (5%) SPEC AXI-5: The ready signal should be asserted within 50/100 cycles after vaild.(Timeout Check1)
- (6%) SPEC AXI-6: The response channel W/B/R should be asserted within 150 cycles after AW/W/AR handsake done.(Timeout Check2)

Note: If any spec is violated, you must show **"SPEC X IS FAIL!"** on your screen. **X is the name of the spec.**

- If multiple specifications are violated at the same time, you can only display the highest priority on the screen, or you will lose "all points" in the demo.
- **The priority of specification:**

  SPEC MAIN-1> SPEC AXI-1 > SPEC AXI-2 > SPEC AXI-3 > SPEC AXI-4 > SPEC AXI-5 > SPEC AXI-6 > SPEC MAIN-2 > SPEC MAIN-3 > SPEC MAIN-4 > SPEC MAIN-5
- After showing "SPEC X IS FAIL!" on your screen, you have to finish your simulation immediately, or you will lose "all points" in the demo.

2. **Design Functionality**: 30% (2nd demo **30% off**)

   Demo: Use our pattern.v and pseudo_DRAM.v, use your HARVESTER.v and DRAM_CTRL.v, and useDRAM_init.dat we provided.

3. **Performance:** 25% (2nd demo **30% off**)
   - **Cycle time ∗ Total latency ∗ Area^2**
   - Area = HARVESTER + DRAM_CTRL
   - The ratio of the pattern number of operation (READ/WRITE/CALC/SORT) is approximately 10:10:30:30.
   - The ranking scores for the first demo and the second demo are separate.

Check the result

Check Test Bench

1. Include HARVESTER _encrypted.v in your TESTBED.v.
2. Check each specification in terminal.

Specification	Command	Check the result on screen
Pattern correctness	*./01_run_vcs_rtl CORRECT*	Congratulations
SPEC MAIN-1	*./01_run_vcs_rtl SPEC_MAIN_1_{1~18}*	SPEC MAIN-1 FAIL
SPEC MAIN-2	*./01_run_vcs_rtl SPEC_MAIN_2_{1~4}*	SPEC MAIN-2 FAIL
SPEC MAIN-3	*./01_run_vcs_rtl SPEC_MAIN_3_{1~6}*	SPEC MAIN-3 FAIL
SPEC MAIN-4	*./01_run_vcs_rtl SPEC_MAIN_4_{1~2}*	SPEC MAIN-4 FAIL
SPEC MAIN-5	*./01_run_vcs_rtl SPEC_MAIN_5_{1~2}*	SPEC MAIN-5 FAIL
SPEC AXI-1	*./01_run_vcs_rtl SPEC_AXI_1_{1~6}*	SPEC AXI-1 FAIL
SPEC AXI -2	*./01_run_vcs_rtl SPEC_ AXI _2_{1~7}*	SPEC AXI -2 FAIL
SPEC AXI -3	*./01_run_vcs_rtl SPEC_ AXI _3_1*	SPEC AXI -3 FAIL
SPEC AXI -4	*./01_run_vcs_rtl SPEC_ AXI _4_{1~2}*	SPEC AXI -4 FAIL
SPEC AXI -5	*./01_run_vcs_rtl SPEC_ AXI _5_{1~5}*	SPEC AXI -5 FAIL
SPEC AXI -6	*./01_run_vcs_rtl SPEC_ AXI 6_{1~3}*	SPEC AXI -6 FAIL

- You can run ***./07_check_pattern*** to check all above specifications.
- Make sure the keyword printed on the screen is EXACTLY THE SAME as the one listed in the table above. You can print out other information according to your preference, but only ONE keyword from the table above is permissible and it is indispensable.

Check Design

1. Include HARVESTER.v in your TESTBED.v.
2. Check each specification in terminal.

Specification	Command	Check the result on screen
Pattern correctness	*./01_run_vcs_rtl*	Congratulations

- You can run ***./08_check_design*** to check the design functionality.

Note

1. Please submit PATTERN.v, HARVESTER.v in **Lab03/EXERCISE/09_SUBMIT**
   a. 1st_demo deadline: 2026/3/23(Mon.) 12:00:00
   b. 2nd_demo deadline: 2026/3/25(Wed.) 12:00:00
2. **Please upload the following file under 09_SUBMIT:**
   - PATTERN.v, Harvester.v, DRAM_CTRL.v
   - If your file **violates the naming rule**, you will **lose 5 points**.
   - Encryption of any uploaded files is prohibited or the demo will fail.
3. **Template folders and reference commands:**
   01_RTL/ (RTL simulation) **./01_run_vcs_rtl**
   02_SYN/ (Synthesis) **./01_run_dc_shell (HARVESTER/DRAM_CTRL)**

   Check if there is any **latch** or **error** in your design in **syn_HARVESTER/DRAM_CTRL.log**
   Check the timing of the design in /Report/**HARVESTERc/DRAM_CTRL.timing**

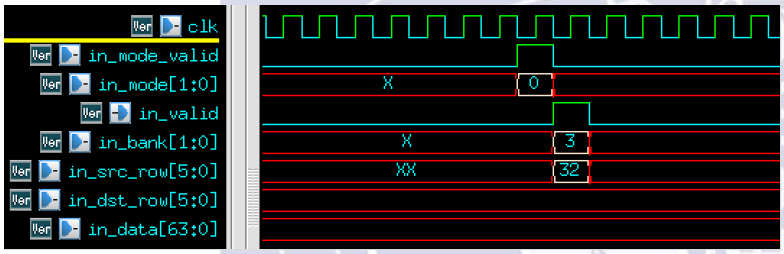
   03_GATE/ (Gate-level simulation) **./01_run_vcs_gate**
   09_SUBMIT (submit files) **./01_submit**
   09_SUBMIT (check files) **./02_check**
   You can key in ./09_clean_up to clear all log files and dump files in each folder.

4. We provide the Harvester_encrypted.v DRAM_CTRL_encrypted.v pseudo_DRAM.v for you. You need to complete the pattern and check the correctness of each design we provided.
5. We provide you with pseudo_DRAM.v and check all the specification related to AXI4-Lite protocol in PATTERN.v.
6. You need to implement the Harvester.v, DRAM_CTRL.v. We only check the functionality using our pattern.
7. Directly access DRAM data in pattern. Use [instance name].[variable name] to access. The variable name in the provided pseudo_DRAM.v is "**DRAM**".
8. In this lab, you need to write a pattern file. You can only use a random system task to generate patterns. You cannot use any IO file to generate patterns. If you submit the txt files you used in your pattern file, this lab will fail.

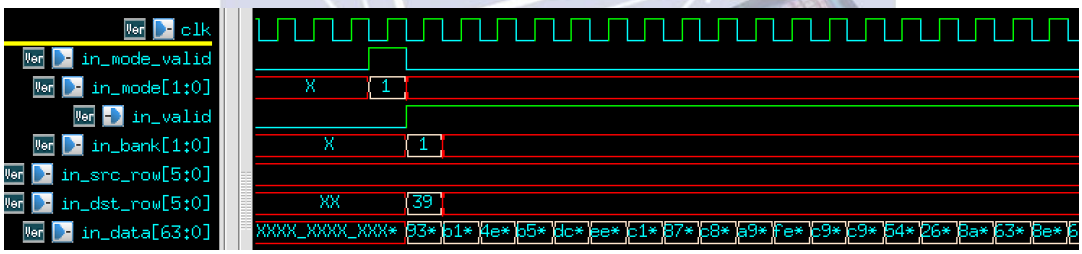
Example Waveform

Input waveform

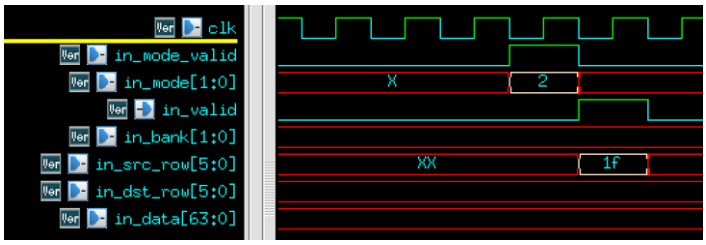
- Mode: READ:



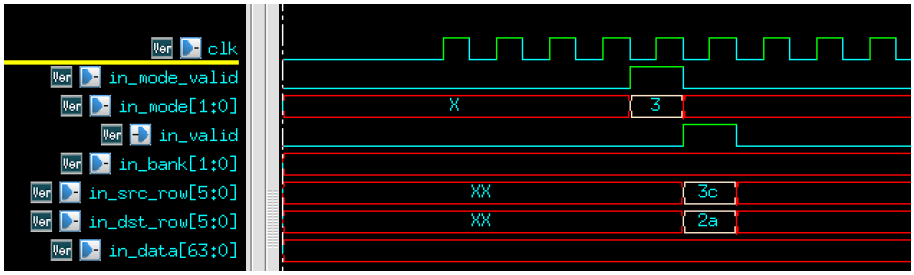
- Mode: WRITE:



- Mode :CALC:

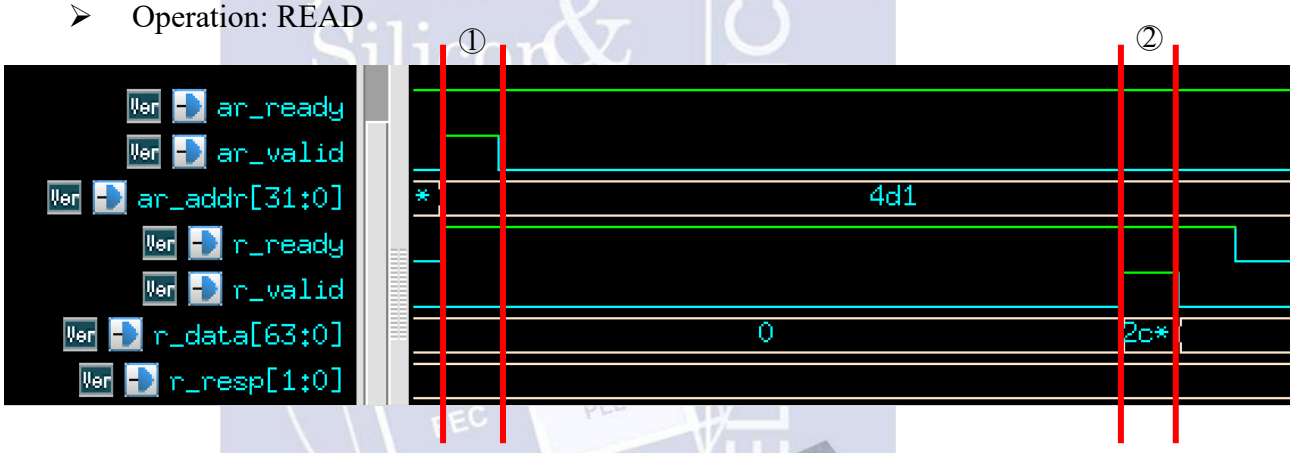


- Mode: SORT:



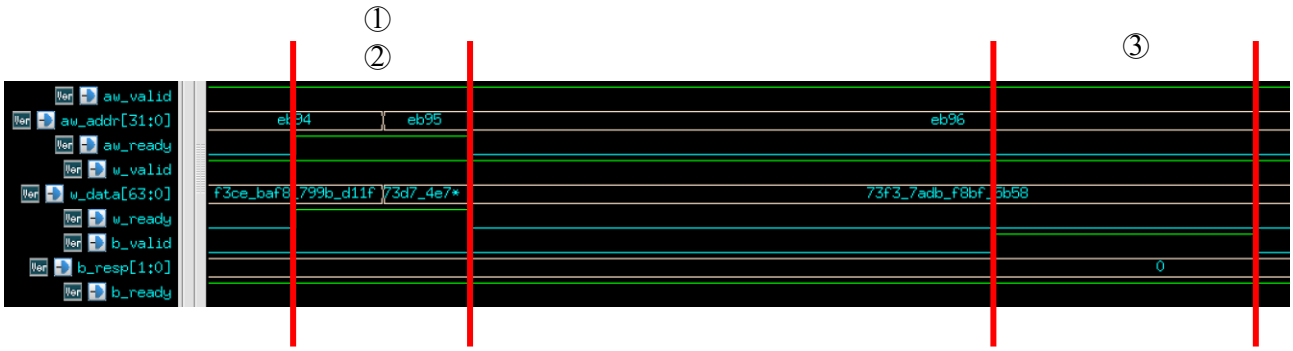
AXI waveform

- Operation: READ



1. Read address channel
2. Read data channel

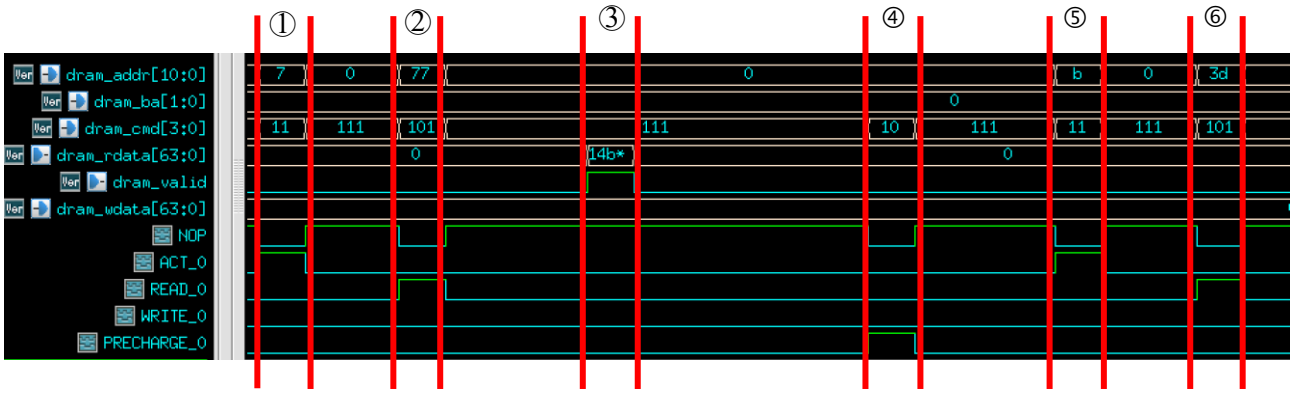
- Operation: WRITE



1. Write address channel
2. Write data channel
3. Write response channel

Physical DRAM interface waveform

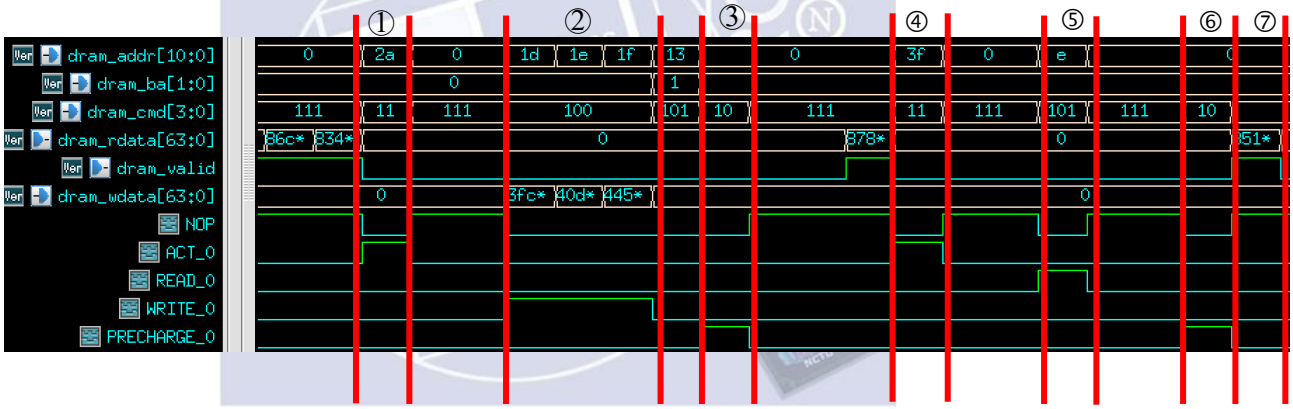
- Operation: READ



1. CMD_ACT: activate {B0_R7}
2. CMD_ READ: read address {B0_C77}

3. DRAM_VALID: read from {B0_R7_C77}
4. CMD_PRE: precharge {B0}
5. CMD_ ACT: activate {B0_RB}
6. CMD_READ: read address {B0_C3D}

- Operation: WRITE



1. CMD_ACT: activate {B0_R2A}
2. CMD_ WRITE: write address {B0_C1D} {B0_1E} {B0_1F} with dram_wdata
3. CMD_PRE: precharge {B0}
4. CMD_ ACT: activate {B0_R3F}
5. CMD_READ: read address {B0_CE}
6. CMD_PRE: precharge {B0}
7. DRAM_VALID: read from {B0_R3F_CE}